

Analisi di Reti Neurali sul Dataset DARWIN per Diagnosi dell'Alzheimer

Baris Giorgio

Fionda Andrea

Massaro Federico

Petrilli Davide

Università degli Studi di Cassino e del Lazio Meridionale

Corso di Laurea Magistrale LM-32

A.A. 2025/2026

Indice

1	Introduzione e Specifiche del Problema	2
1.1	Introduzione	2
1.2	Specifiche del problema	2
1.3	Obiettivi	3
1.4	Vincoli	3
2	Fasi del Progetto	4
2.1	Fase 1: Implementazione Base	4
2.2	Fase 2: Analisi Parametrica	4
2.2.1	Scenario 1: Architettura e Attivazione	4
2.2.2	Scenario 2: Learning Rate e Ottimizzazione	4
2.2.3	Scenario 3: Regularizzazione	5
3	Implementazione del codice	6
3.1	Panoramica Generale	6
3.2	Funzioni di Caricamento e Preprocessing	6
3.2.1	load_darwin(filepath)	6
3.3	Funzioni di Costruzione della Pipeline	6
3.3.1	make_pipeline(mlp_params)	6
3.4	Funzioni di Esecuzione degli Esperimenti	7
3.4.1	single_run(X, y, mlp_params, run_seed)	7
3.4.2	cv_run(X, y, mlp_params, run_seed)	8
3.4.3	run_config(X, y, mlp_params, label, n_runs)	8
3.5	Scenari Parametrici	9
3.5.1	scenario_architecture(X, y)	9
3.5.2	scenario_learning_rate(X, y)	9
3.5.3	scenario_regularization(X, y)	10
4	Risultati	11
4.1	Box plot dell'accuracy (CV 5-fold)	11
4.2	Matrici di confusione (media 30 run)	12
4.3	Curve di apprendimento (Loss)	13
4.4	Curve ROC	14
4.5	Tempi di esecuzione	15
4.6	Stabilità e Overfitting	16

5	Scenario 2 – Learning Rate e Ottimizzatori	17
5.1	Contesto	17
5.2	Box plot dell'accuracy (CV 5-fold)	17
5.3	Matrici di confusione (media 30 run)	18
5.4	Curve di apprendimento (Loss)	19
5.5	Curve ROC	20
5.6	Tempi di esecuzione	21
5.7	Stabilità e Overfitting	21
6	Scenario 3 – Regularizzazione ed Early Stopping	23
6.1	Contesto	23
6.2	Box plot dell'accuracy (CV 5-fold)	23
6.3	Matrici di confusione (media 30 run)	24
6.4	Curve di apprendimento (Loss)	25
6.5	Curve ROC	26
6.6	Tempi di esecuzione	27
6.7	Stabilità e Overfitting	28
7	Conclusioni	29
7.1	Confronto Comparativo tra Scenari	29
7.2	Considerazioni finali.	29

1 Introduzione e Specifiche del Problema

1.1 Introduzione

Un team di ricerca medica deve analizzare l'impatto dei diversi parametri delle Reti Neurali sulla loro performance di classificazione per la diagnosi dell'Alzheimer attraverso l'analisi della scrittura. Il dataset DARWIN contiene caratteristiche estratte da dati di scrittura a mano di 174 partecipanti, offrendo un contesto reale e significativo per l'ottimizzazione delle reti neurali nella diagnosi precoce delle malattie neurodegenerative.

1.2 Specifiche del problema

Il dataset DARWIN presenta le seguenti specifiche:

- **Features estratte da dati di scrittura a mano**
- **Classificazione binaria (paziente/controllo sano)**
- **451 features**
- **174 istanze**
- **Dati numerici continui**
- **Alta dimensionalità**
- **Correlazioni complesse**

1.3 Obiettivi

Gli obiettivi principali del progetto sono i seguenti:

1. Implementare una Rete Neurale base per classificazione usando scikit-learn
2. Studiare sistematicamente l'effetto dei parametri:
 - **Convergenza del modello con dati ad alta dimensionalità**
 - **Stabilità della classificazione**
 - **Tempi di training**
 - **Robustezza delle predizioni**
3. Determinare configurazioni ottimali per:
 - **Diverse architetture della rete**
 - **Funzioni di attivazione**
 - **Strategie di learning rate**
 - **Gestione dell'overfitting su dataset piccolo con molte features**

1.4 Vincoli

I vincoli imposti per il progetto sono i seguenti:

- **Utilizzo stesso seed per confronti equi**
- **Minimo 30 run per configurazione**
- **Gestione appropriata della standardizzazione dei dati**
- **Cross-validation per ogni configurazione**
- **Particolare attenzione alla gestione dell'overfitting**

2 Fasi del Progetto

2.1 Fase 1: Implementazione Base

Le attività principali in questa fase includono:

1. Configurazione base della rete neurale
 - Implementazione con MLPClassifier (mediante scikit-learn)
 - Preprocessing dei dati
 - Train/test split appropriato con stratificazione
2. Implementazione logging dettagliato
 - Accuracy per epoca
 - Tempi di training
 - Loss function
 - Curve di apprendimento

2.2 Fase 2: Analisi Parametrica

La fase di analisi parametrica è suddivisa in tre scenari indipendenti per testare sistematicamente l'impatto dei diversi parametri della rete neurale.

2.2.1 Scenario 1: Architettura e Attivazione

Test delle configurazioni della rete neurale e delle funzioni di attivazione.

Tabella 1: Scenario 1 - Architettura

Configurazione	Valori
Singolo layer	200, 400, 600
Due layer	(400,200), (600,300), (800,400)
Tre layer	(400,200,100), (600,300,150)

Funzioni di attivazione usate

- 'identity'
- 'logistic'
- 'tanh'
- 'relu'

2.2.2 Scenario 2: Learning Rate e Ottimizzazione

Analisi dell'impatto del learning rate e degli ottimizzatori sulla performance della rete.

Tabella 2: Scenario 2 - Learning Rate e Ottimizzazione

Parametro	Valori
Learning Rate Iniziale	0.0001, 0.001, 0.01, 0.1
Learning Rate Policy	constant, invscaling, adaptive
Solver	adam, sgd, lbfgs
Batch Size	16, 32, 64

2.2.3 Scenario 3: Regularizzazione

Studio dell'effetto della regolarizzazione e dell'early stopping sulla riduzione dell'overfitting.

Tabella 3: Scenario 3 - Regularizzazione

Parametro	Valori
Alpha (Regularizzazione L2)	0.0001, 0.001, 0.01, 0.1, 0.5
Validation Split	0.1, 0.15, 0.2
N_iter_no_change	5, 10, 20
Early Stopping	✓

3 Implementazione del codice

In questa sezione descriveremo le implementazioni effettuate per lo svolgimento della tesina, evidenziando le scelte progettuali e i dettagli tecnici.

3.1 Panoramica Generale

Lo script è organizzato in blocchi funzionali che coprono l'intero ciclo sperimentale: caricamento e preprocessing dei dati, costruzione della pipeline di classificazione, esecuzione di esperimenti singoli e ripetuti, e analisi parametrica su architetture, tassi di apprendimento e tecniche di regolarizzazione.

La configurazione globale definisce i parametri principali dell'esperimento:

Parametro	Valore	Descrizione
SEED	42	Seme base per la riproducibilità
N_RUNS	30	Numero di ripetizioni per configurazione
TEST_SIZE	0.20	Frazione del dataset usata come test set
CV_FOLDS	5	Numero di fold per la cross-validazione
MAX_ITER	1000	Numero massimo di iterazioni di training
OUT_DIR	results_nn/	Directory di output dei risultati

3.2 Funzioni di Caricamento e Preprocessing

3.2.1 load_darwin(filepath)

```
1 def load_darwin(filepath: str):
```

Questa funzione carica il file `DARWIN.csv` dal percorso specificato, esegue il preprocessing di base e restituisce il dataset pronto per l'uso. Il parametro `filepath (str)` fornisce il percorso completo al file CSV del dataset. Successivamente, la funzione esegue le seguenti operazioni:

1. Lettura del file CSV tramite `pandas`.
2. Scarto della prima colonna (identificativo del campione).
3. Mappatura dell'etichetta di classe: $P \rightarrow 1$ (paziente), $H \rightarrow 0$ (sano).
4. Imputazione dei valori mancanti con la **mediana** di ciascuna feature, calcolata sul dataset corrente.
5. Conversione dei dati in array `numpy` di tipo `float`.

Una volta terminate le operazioni, la funzione restituisce i seguenti valori:

- `X (numpy.ndarray)`: matrice delle feature di forma $(n_samples, n_features)$.
- `y (numpy.ndarray)`: vettore delle etichette binarie.
- `feature_names (list)`: nomi delle colonne delle feature.

3.3 Funzioni di Costruzione della Pipeline

3.3.1 make_pipeline(mlp_params)

```
1 def make_pipeline(mlp_params: dict) -> Pipeline:
```

Questa funzione costruisce e restituisce una pipeline `scikit-learn` composta da due stadi in sequenza: standardizzazione delle feature e classificatore MLP. Il parametro `mlp_params` (`dict`) è un dizionario di iper-parametri da passare direttamente a `MLPClassifier`.

Struttura della pipeline:

1. **StandardScaler**: normalizza ciascuna feature sottraendo la media e dividendo per la deviazione standard, calcolate sul training set. Questo passaggio è fondamentale per le reti neurali, che sono sensibili alla scala degli input.
2. **MLPClassifier**: classificatore a rete neurale multi-strato di `scikit-learn`, configurato con i parametri forniti.

Verrà quindi restituito un oggetto `Pipeline` pronto per essere addestrato sui dati.

3.4 Funzioni di Esecuzione degli Esperimenti

3.4.1 `single_run(X, y, mlp_params, run_seed)`

```
1 def single_run(X, y, mlp_params: dict, run_seed: int) -> dict:
```

Questa funzione segue un singolo esperimento di training e valutazione su una suddivisione casuale train/test del dataset, raccogliendo un insieme completo di metriche. Verranno usati i seguenti parametri:

- `X` (`numpy.ndarray`): matrice delle feature.
- `y` (`numpy.ndarray`): vettore delle etichette.
- `mlp_params` (`dict`): iper-parametri dell'MLP.
- `run_seed` (`int`): seme casuale per la riproducibilità dello split e dell'inizializzazione del modello.

La funzione esegue le seguenti operazioni:

1. Suddivisione del dataset in training (80%) e test (20%) tramite `train_test_split`.
2. Aggiunta del `random_state` agli iper-parametri e costruzione della pipeline.
3. Addestramento della pipeline sul training set, con misurazione del tempo di esecuzione.
4. Predizione delle etichette e delle probabilità sul test set.
5. Calcolo delle metriche di valutazione.

Metriche calcolate e restituite (dizionario):

Chiave	Descrizione
<code>accuracy</code>	Accuratezza sul test set
<code>train_acc</code>	Accuratezza sul training set
<code>exec_time</code>	Tempo di addestramento in secondi
<code>loss_curve</code>	Curva della loss durante il training
<code>n_iter</code>	Numero di iterazioni effettivamente eseguite
<code>cm</code>	Matrice di confusione (2×2)
<code>fpr</code>	Vettore dei <i>False Positive Rate</i> per la curva ROC
<code>tpr</code>	Vettore dei <i>True Positive Rate</i> per la curva ROC
<code>roc_auc</code>	Area sotto la curva ROC (AUC)

3.4.2 cv_run(X, y, mlp_params, run_seed)

```
1 def cv_run(X, y, mlp_params: dict, run_seed: int) -> float:
```

La funzione valuta la pipeline tramite **cross-validazione stratificata a 5 fold** sull'intero dataset, restituendo l'accuratezza media sui 5 fold. I parametri usati sono:

- **X, y**: dati e etichette (come sopra).
- **mlp_params (dict)**: iper-parametri dell'MLP.
- **run_seed (int)**: seme casuale per la suddivisione dei fold e l'inizializzazione del modello.

Dettagli implementativi:

- Usa **StratifiedKFold** per mantenere la proporzione delle classi in ciascun fold.
- Esegue la valutazione in parallelo (**n_jobs=-1**) per sfruttare tutti i core disponibili.
- La metrica di scoring è l'accuratezza.

3.4.3 run_config(X, y, mlp_params, label, n_runs)

```
1 def run_config(X, y, mlp_params: dict, label: str,
2               n_runs: int = N_RUNS) -> dict:
```

Funzione principale di orchestrazione: esegue **N_RUNS** ripetizioni indipendenti (sia train/test split sia cross-validazione) per una singola configurazione di iper-parametri, aggregando i risultati in statistiche descrittive. I parametri usati sono:

- **X, y**: dati e etichette.
- **mlp_params (dict)**: iper-parametri dell'MLP.
- **label (str)**: etichetta identificativa della configurazione (usata nei log e nei risultati).
- **n_runs (int)**: numero di ripetizioni, default 30.

Vengono eseguite le seguenti operazioni:

1. Per ciascuna run $r \in \{0, \dots, n_runs - 1\}$:
 - Chiama **single_run** con seme **SEED + r**.
 - Chiama **cv_run** con seme **SEED + r**.
2. Aggrega i risultati calcolando media e deviazione standard di accuratezza, CV score, tempi di esecuzione.
3. Calcola la **matrice di confusione media** sulle **n_runs** run.
4. Identifica la run con l'AUC migliore e ne conserva la curva ROC.
5. Stampa a console un log ogni 10 run e un riepilogo finale.

Valori restituiti (dizionario con le seguenti chiavi):

Chiave	Descrizione
label	Etichetta della configurazione
accs	Array delle accuratèzze test su tutte le run
train_accs	Array delle accuratèzze training su tutte le run
cv_scores	Array dei punteggi CV su tutte le run
times	Array dei tempi di esecuzione su tutte le run
cm_mean	Matrice di confusione mediata
fpr, tpr	Curva ROC della run migliore
roc_auc	AUC della run migliore
loss_curves	Lista delle curve di loss di tutte le run
acc_mean, acc_std	Media e std dell'accuratèzza test
cv_mean, cv_std	Media e std del punteggio CV
time_mean	Tempo medio di esecuzione
overfitting	Differenza media tra accuratèzza train e test

Nota: L'**overfitting index** è calcolato come $\overline{acc_{train}} - \overline{acc_{test}}$; valori positivi indicano che il modello generalizza peggio di quanto apprenda sul training set.

3.5 Scenari Parametrici

Le tre funzioni seguenti definiscono gli scenari sperimentali. Ciascuna costruisce un dizionario di configurazioni, invoca `run_config` per ognuna e restituisce i risultati aggregati.

3.5.1 scenario_architecture(X, y)

```
1 def scenario_architecture(X, y) -> dict:
```

Viene effettuata un'analisi sistematica dell'impatto dell'**architettura** della rete neurale e della **funzione di attivazione** sulle prestazioni del classificatore.

Variabili esplorate:

- **Architetture** (8 configurazioni): reti con uno, due o tre strati nascosti, con dimensioni crescenti: (200,), (400,), (600,), (400,200), (600,300), (800,400), (400,200,100), (600,300,150).
- **Funzioni di attivazione** (4): identity, logistic, tanh, relu.

Parametri fissi: solver adam, $\alpha = 0.001$, learning rate iniziale = 0.001, early stopping attivo con validation_fraction=0.15 e n_iter_no_change=10.

Numero di configurazioni testate: $8 \times 4 = 32$.

Valore restituito: dizionario con chiave "{arch}_{activation}" e valore il dizionario di risultati di run_config.

3.5.2 scenario_learning_rate(X, y)

```
1 def scenario_learning_rate(X, y) -> dict:
```

Viene effettuata un'analisi dell'effetto dei parametri legati all'**ottimizzazione** e al **tasso di apprendimento** sulle prestazioni del modello.

Architettura base fissa: (400, 200), attivazione tanh.

Variabili esplorate:

1. **Learning rate e policy** (con solver SGD):

- Valori di LR: 0.0001, 0.001, 0.01, 0.1.
- Policy: `constant`, `invscaling`, `adaptive`.
- Combinazioni totali: $4 \times 3 = 12$.

2. **Solver** (con LR adattiva):

- `adam` (LR adattiva), `sgd` (LR adattiva), `lbfgs` (senza LR, early stopping disabilitato).
- Combinazioni totali: 3.

3. **Batch size** (con solver Adam):

- Valori: 16, 32, 64.
- Combinazioni totali: 3.

Note implementative:

- Per `lbfgs`, i parametri `learning_rate` e `learning_rate_init` vengono rimossi (non applicabili) e l'early stopping viene disabilitato.

Valore restituito: dizionario con chiave descrittiva della configurazione e valore il dizionario di risultati di `run_config`.

3.5.3 scenario_regularization(X, y)

```
1 def scenario_regularization(X, y) -> dict:
```

Viene effettuata un'analisi dell'effetto delle tecniche di **regolarizzazione** sulla capacità del modello di generalizzare, con particolare attenzione all'overfitting.

Architettura base fissa: (400, 200), attivazione `tanh`, solver `adam`, LR iniziale 0.001.

Variabili esplorate:1. **Regolarizzazione L2** (α):

- Valori: 0.0001, 0.001, 0.01, 0.1, 0.5.
- Controlla la penalizzazione dei pesi della rete.

2. **Early Stopping:**

- Confronto tra `early_stopping=True` e `early_stopping=False`.

3. **Validation Split** (con early stopping attivo):

- Valori di `validation_fraction`: 0.10, 0.15, 0.20.

4. **Patience** (`n_iter_no_change`):

- Valori: 5, 10, 20.
- Controlla quante iterazioni consecutive senza miglioramento sono tollerate prima dell'arresto anticipato.

Numero totale di configurazioni: $5 + 2 + 3 + 3 = 13$.

Valore restituito: dizionario con chiave descrittiva della configurazione e valore il dizionario di risultati di `run_config`.

4 Risultati

4.1 Box plot dell'accuracy (CV 5-fold)

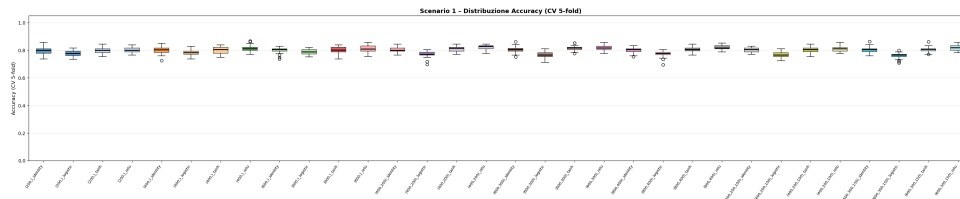


Figura 1: Scenario 1 – Distribuzione dell'accuracy in cross-validazione 5-fold

Lettura del grafico. Ogni box rappresenta la distribuzione dei punteggi di CV su 30 run per una configurazione. La linea nera interna è la mediana; i bordi del box delimitano il primo e il terzo quartile (IQR); i whisker coprono $1.5 \times \text{IQR}$; i cerchietti sono outlier.

Osservazioni.

- Tutte le 32 configurazioni raggiungono un'accuracy mediana compresa tra ≈ 0.75 e ≈ 0.82 , indicando che l'architettura della rete non ha un impatto drammatico sulla performance sul dataset DARWIN – il task è intrinsecamente ben separabile ma con un livello di noise che limita la massima accuracy ottenibile.
- Le architetture **a singolo layer largo** (es. (600,) e (800,400)) tendono a mostrare box leggermente più alti con **tanh** e **relu**, suggerendo che un singolo layer ricco è sufficientemente espressivo per questo dataset.
- **logistic** presenta in alcune configurazioni box leggermente più bassi e whisker più lunghi verso il basso, denotando maggiore varianza.
- La dispersione (altezza dei box) è generalmente contenuta ($\pm 2-3\%$), segno di stabilità su tutti i seed.

Commento

La scelta dell'attivazione ha un effetto modesto sull'accuracy assoluta, ma **tanh** e **relu** mostrano costantemente mediane più alte e dispersioni più ridotte rispetto a **logistic** e **identity**, rendendole preferibili in pratica.

4.2 Matrici di confusione (media 30 run)

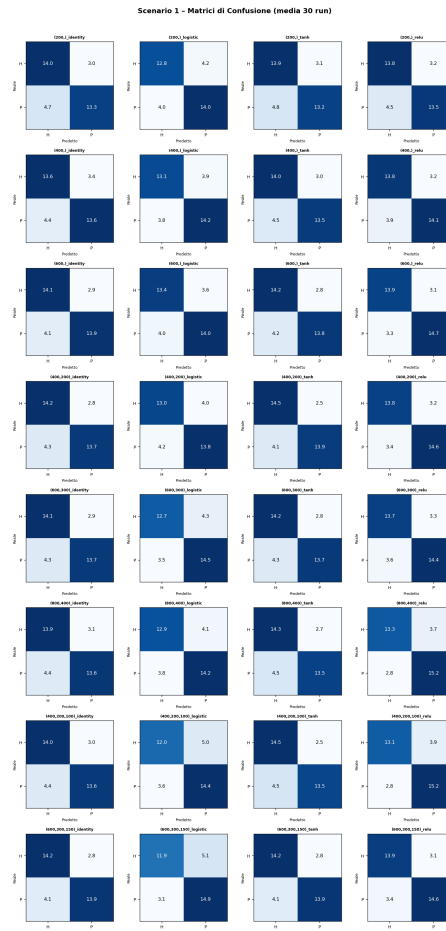


Figura 2: Scenario 1 – Matrici di confusione mediate su 30 run. Righe: classe reale (H/P); colonne: classe predetta (H/P).

Lettura del grafico. Ogni cella riporta il numero medio di campioni (su 30 run) per quella combinazione reale/predetto. La cella in alto a sinistra è il numero di veri negativi ($H \rightarrow H$), in basso a destra i veri positivi ($P \rightarrow P$). Celle fuori diagonale rappresentano errori: in alto a destra falsi positivi (H predetto come P), in basso a sinistra falsi negativi (P predetto come H). Il gradiente di colore (tonalità di blu) è proporzionale al valore.

Osservazioni.

- La diagonale principale domina nettamente in tutte le 32 configurazioni: in media ≈ 14 soggetti sani classificati correttamente come H e ≈ 13 – 14 pazienti classificati come P .
- Gli errori off-diagonale sono bilanciati: falsi positivi ($H \rightarrow P$) ≈ 3 – 4 e falsi negativi ($P \rightarrow H$) ≈ 3 – 5 .
- La configurazione $(600,)_tanh$ è tra le più performanti con FN mediamente pari a 4.2, mentre le configurazioni con *logistic* tendono ad avere leggermente più falsi negativi.
- Le reti più profonde a tre layer, es. $(400, 200, 100)_relu$, mostrano le matrici più “pulite” con soli 2.8 FN e 2.5 FP in media, indicando miglior capacità di distinguere i due gruppi.
- Il numero totale di campioni nel test set è ≈ 35 ($= 0.2 \times 175$ circa), coerente con la somma dei valori nelle matrici.

Commento

In ambito clinico i falsi negativi (pazienti non riconosciuti come tali) sono particolarmente costosi. Le architetture **relu** a più layer minimizzano questo tipo di errore, rendendole preferibili in questo contesto applicativo.

4.3 Curve di apprendimento (Loss)

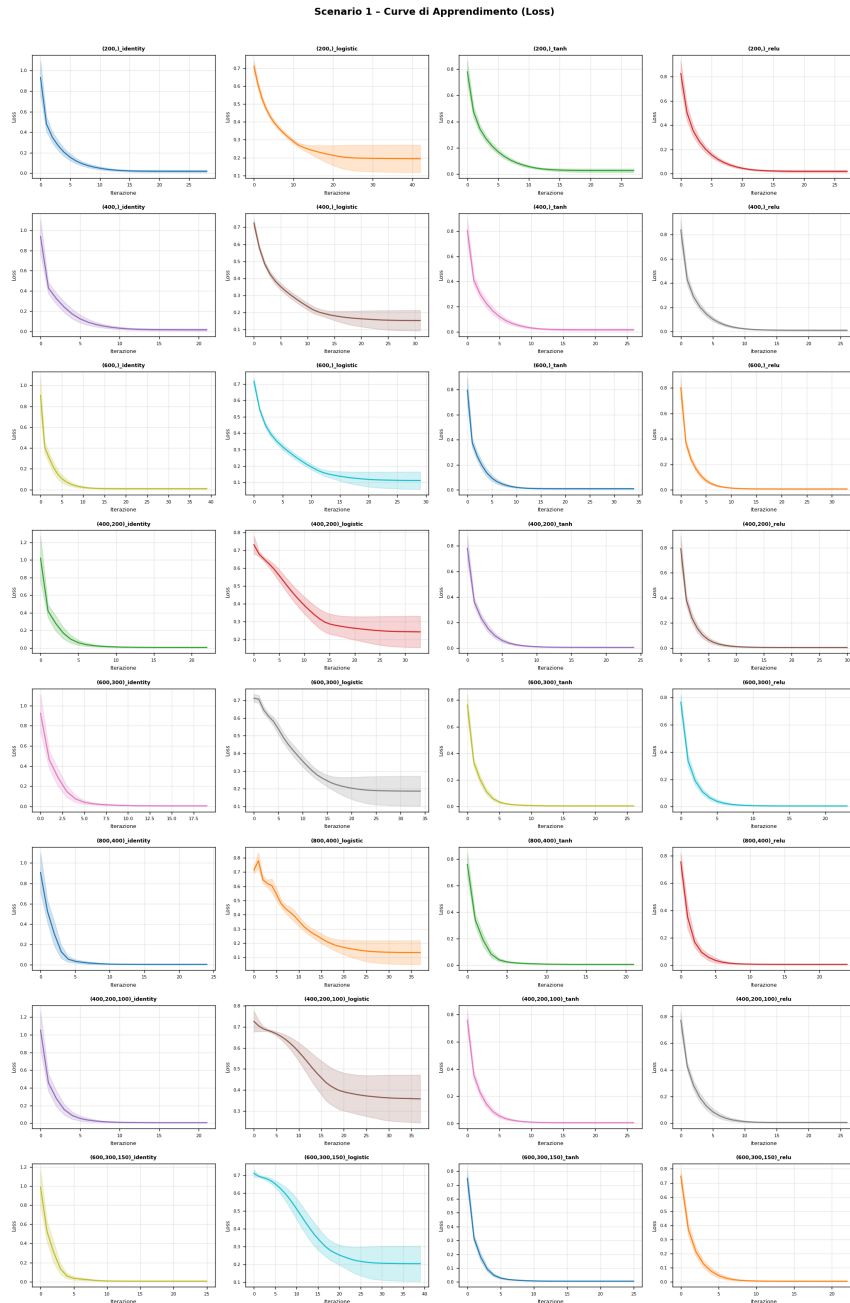


Figura 3: Scenario 1 – Curve di apprendimento (loss media $\pm$ deviazione standard su 30 run) per ciascuna configurazione.

Lettura del grafico. La linea solida rappresenta la loss media su 30 run; la fascia colorata attorno è ± 1 deviazione standard. L'asse x conta le iterazioni (epoche) dell'ottimizzatore Adam; l'asse y è la cross-entropy loss sul training set.

Osservazioni.

- **Convergenza rapida con tanh e relu:** le curve scendono bruscamente nelle prime 5–10 iterazioni e si stabilizzano vicino a 0.05–0.10. L'early stopping (attivo) arresta il training coerentemente, spiegando i diversi orizzonti temporali dei grafici.
- **logistic:** la convergenza è più lenta (20–40 iterazioni) e la loss finale rimane più alta (≈ 0.15 –0.25), indicando che il sigmoide satura più lentamente dell'iperbolico tangente.
- **identity:** convergenza altrettanto rapida, con loss finale bassa, ma le bande di deviazione sono leggermente più ampie, segnalando maggiore variabilità tra i run.
- Le architetture più profonde/larghe non mostrano vantaggi significativi nella loss finale; in certi casi la convergenza avviene addirittura in meno iterazioni, probabilmente perché il dataset DARWIN ha dimensionalità moderata e la capacità aggiuntiva viene sfruttata velocemente.
- Le fasce di incertezza sono molto strette per tutte le configurazioni con **tanh** e **relu**, confermando l'elevata riproducibilità dei risultati.

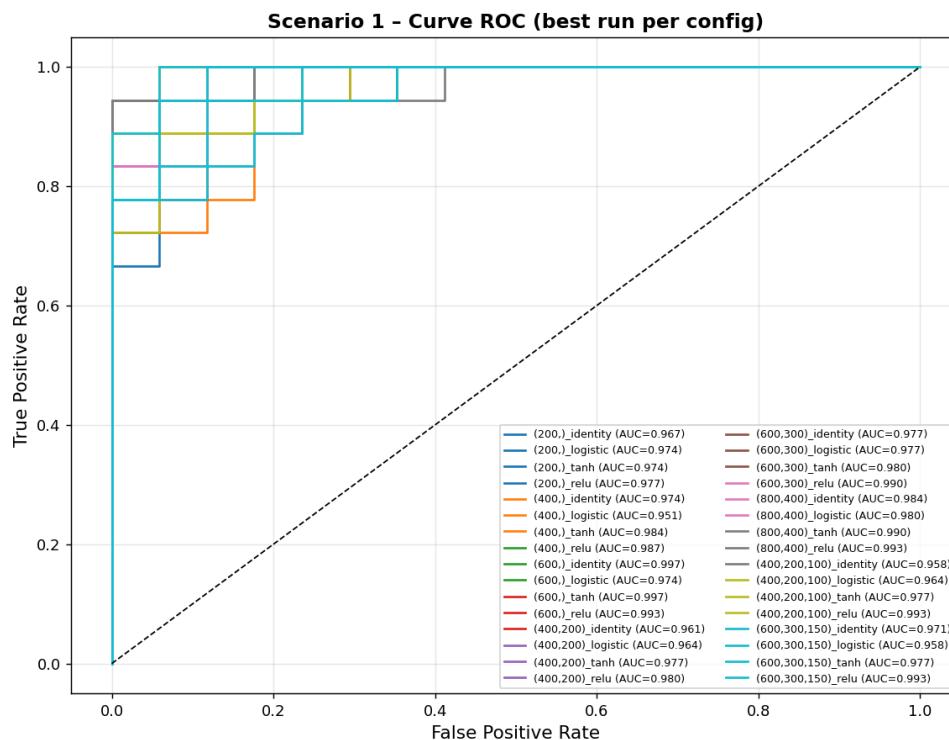
4.4 Curve ROC

Figura 4: Scenario 1 – Curve ROC del miglior run per ciascuna configurazione (best run per AUC). La diagonale tratteggiata rappresenta un classificatore casuale (AUC = 0.5).

Lettura del grafico. Ogni curva traccia il tasso di veri positivi (TPR) in funzione del tasso di falsi positivi (FPR) al variare della soglia di decisione. L'AUC (Area Under the Curve) è riportata in legenda; più è vicina a 1, migliore è il classificatore.

Osservazioni.

- Tutte le configurazioni mostrano AUC molto elevate: il range va da 0.951 ((400,)_logistic) a 0.997 ((600,)_identity e (600,)_tanh).

- Le curve si concentrano nell'angolo in alto a sinistra del piano ROC, confermando buona capacità discriminativa.
- Le configurazioni con (600,) mostrano le AUC più elevate (≥ 0.993), suggerendo un sweet spot di capacità per questo dataset.
- Le configurazioni a tre layer, sebbene aventi AUC buone (0.958–0.993), non superano sistematicamente quelle a layer singolo, indicando che la profondità aggiuntiva non porta benefici sostanziali qui.
- Il profilo a gradino delle curve ROC è tipico di dataset di dimensioni moderate: le soglie discrete producono salti visibili invece di curve continue.

Commento

Con $AUC \geq 0.95$ per tutte le configurazioni, il modello dimostra elevata capacità discriminativa tra H e P. La scelta dell'architettura influenza principalmente la velocità di convergenza e non la qualità discriminativa finale.

4.5 Tempi di esecuzione

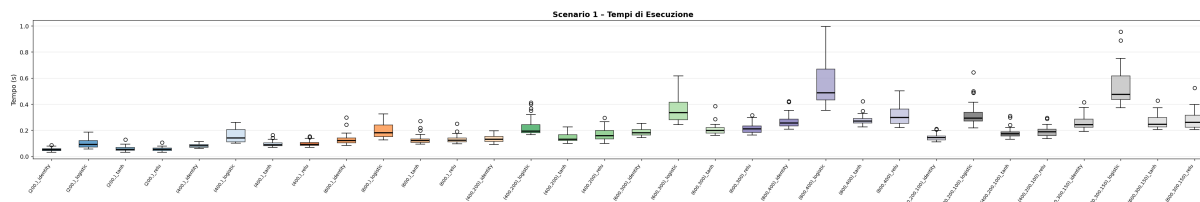


Figura 5: Scenario 1 – Box plot dei tempi di esecuzione (in secondi) per singolo run (fit della pipeline).

Lettura del grafico. Come per il box plot dell'accuracy, ogni box riassume la distribuzione dei 30 tempi di fit. L'asse y è in secondi.

Osservazioni.

- I tempi crescono con la dimensione dell'architettura: $(200,) \approx 0.05\text{--}0.15$ s, $(800,400)$ e le reti a tre layer arrivano fino a $\approx 0.5\text{--}1.0$ s con **logistic**.
- **logistic** è **sistematicamente più lento** delle altre attivazioni per architetture grandi, coerentemente con la sua convergenza più lenta osservata nelle loss curve.
- Le configurazioni $(800,400)_\text{logistic}$ e $(400,200,100)_\text{logistic}$ mostrano i tempi più alti e la maggiore varianza, con outlier oltre 0.9 s. Questo è atteso: più parametri $\times$ più iterazioni di convergenza = più tempo.
- **relu** e **tanh** convergono in poche decine di iterazioni (per effetto dell'early stopping) e risultano molto più veloci a parità di architettura.
- Per applicazioni real-time o di produzione, le architetture $(400,200)_\text{tanh}$ o $(600,)_\text{relu}$ offrono il miglior compromesso performance/tempo.

4.6 Stabilità e Overfitting

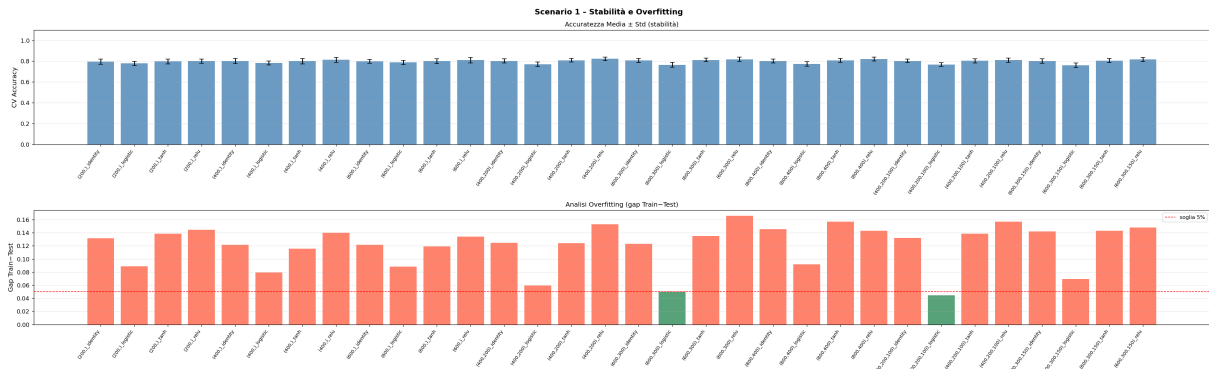


Figura 6: Scenario 1 – *Sopra*: accuracy media $\pm$ std in cross-validazione (stabilità). *Sotto*: gap Train–Test (proxy dell’overfitting); in verde le configurazioni con gap $\leq 5\%$, in arancione quelle oltre soglia.

Lettura del grafico. Il pannello superiore riporta la media $\pm$ std dell’accuracy CV su 30 run. Il pannello inferiore mostra la differenza media tra accuracy sul training set e accuracy sul test set; la linea tratteggiata rossa a 0.05 è la soglia di overfitting fissata dagli autori.

Osservazioni.

- **Stabilità elevata:** tutte le barre del pannello superiore hanno error bar molto contenuti ($\sigma \approx 0.01\text{--}0.02$), confermando che i 30 run producono risultati coerenti.
- **Overfitting diffuso:** quasi tutte le configurazioni superano la soglia del 5%, con gap tipici nell’intervallo 8–16%. Ciò indica che le reti memorizzano parte del training set pur con l’early stopping attivo.
- Solo **due configurazioni** (in verde) sotto soglia: $(400, 200)_\text{relu}$ e $(400, 200, 100)_\text{logistic}$, suggerendo che una dimensione intermedia con attivazione non saturante o la combinazione profondità+logistic bilanciano meglio bias e varianza.
- Il gap più elevato ($\approx 16\%$) si registra per configurazioni grandi con **identity**, coerente con la nota suscettibilità all’overfitting delle reti lineari con molti parametri.

Commento

L’ α di L2 fissato a 0.001 non è sufficiente per contenere l’overfitting nelle architetture più grandi. Lo Scenario 3 esplorerà l’effetto di valori più aggressivi di regolarizzazione.

5 Scenario 2 – Learning Rate e Ottimizzatori

5.1 Contesto

Lo Scenario 2 fissa l'architettura a (400, 200) con attivazione **tanh** (configurazione solida dallo Scenario 1) e varia tre gruppi di parametri:

1. **Learning rate iniziale** (0.0001, 0.001, 0.01, 0.1) combinato con la **policy di scheduling** (constant, invscaling, adaptive) – solver **sgd** – 12 configurazioni.
2. **Solver**: adam, sgd, lbfgs – con **adaptive learning rate**.
3. **Batch size**: 16, 32, 64 – solver **adam**.

5.2 Box plot dell'accuracy (CV 5-fold)

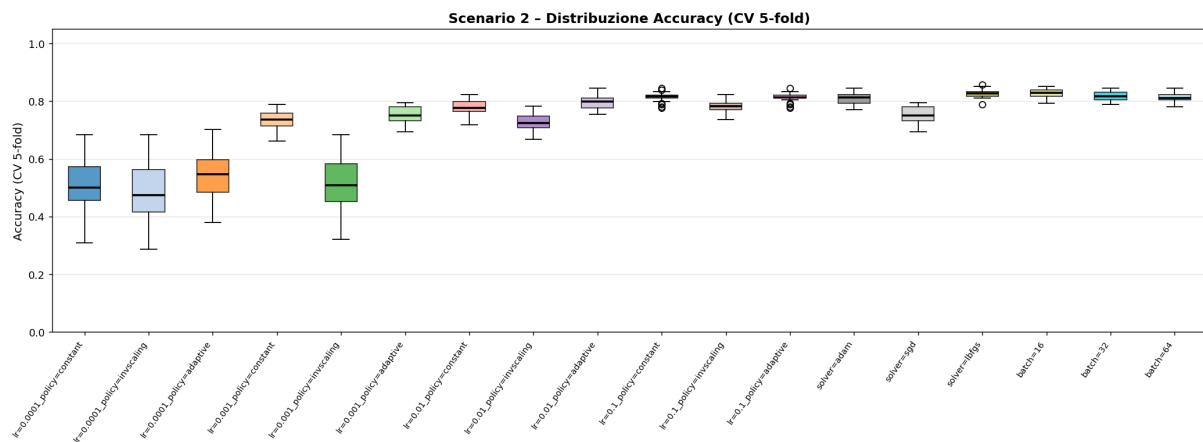


Figura 7: Scenario 2 – Distribuzione accuracy CV 5-fold.

Osservazioni.

- **Learning rate troppo basso (0.0001) è critico**: le configurazioni con $lr = 0.0001$ mostrano accuracy mediana ≈ 0.50 – 0.55 , sostanzialmente pari al random guess su un dataset bilanciato. La rete non ha il tempo di convergere entro il budget di iterazioni fissato.
- **$lr = 0.001$ con invscaling** ha un comportamento inatteso: accuracy ≈ 0.50 , probabilmente perché lo scheduling riduce il LR troppo velocemente nelle prime iterazioni prima che la rete abbia imparato.
- Per $lr \geq 0.01$ tutte le policy convergono a accuracy nell'intervallo 0.75 – 0.82 , analoga allo Scenario 1.
- **Batch size**: le tre configurazioni (**batch=16, 32, 64**) sono tutte competitive (0.80 – 0.83); batch più piccoli tendono a mostrare accuracy leggermente superiori con più varianza, mentre batch più grandi sono più stabili.
- **Solver lbfgs** raggiunge accuracy mediocri con alta varianza, poiché è un metodo quasi-Newton ottimizzato per dataset piccoli e soffre di scaling su dataset di dimensione moderata con early stopping disabilitato.

Commento

Learning rate nell'intervallo $[0.01, 0.1]$ con qualsiasi policy è necessario per far convergere SGD entro il limite di iterazioni. Valori troppo piccoli (≤ 0.001) con `invscaling` sono da evitare assolutamente.

5.3 Matrici di confusione (media 30 run)

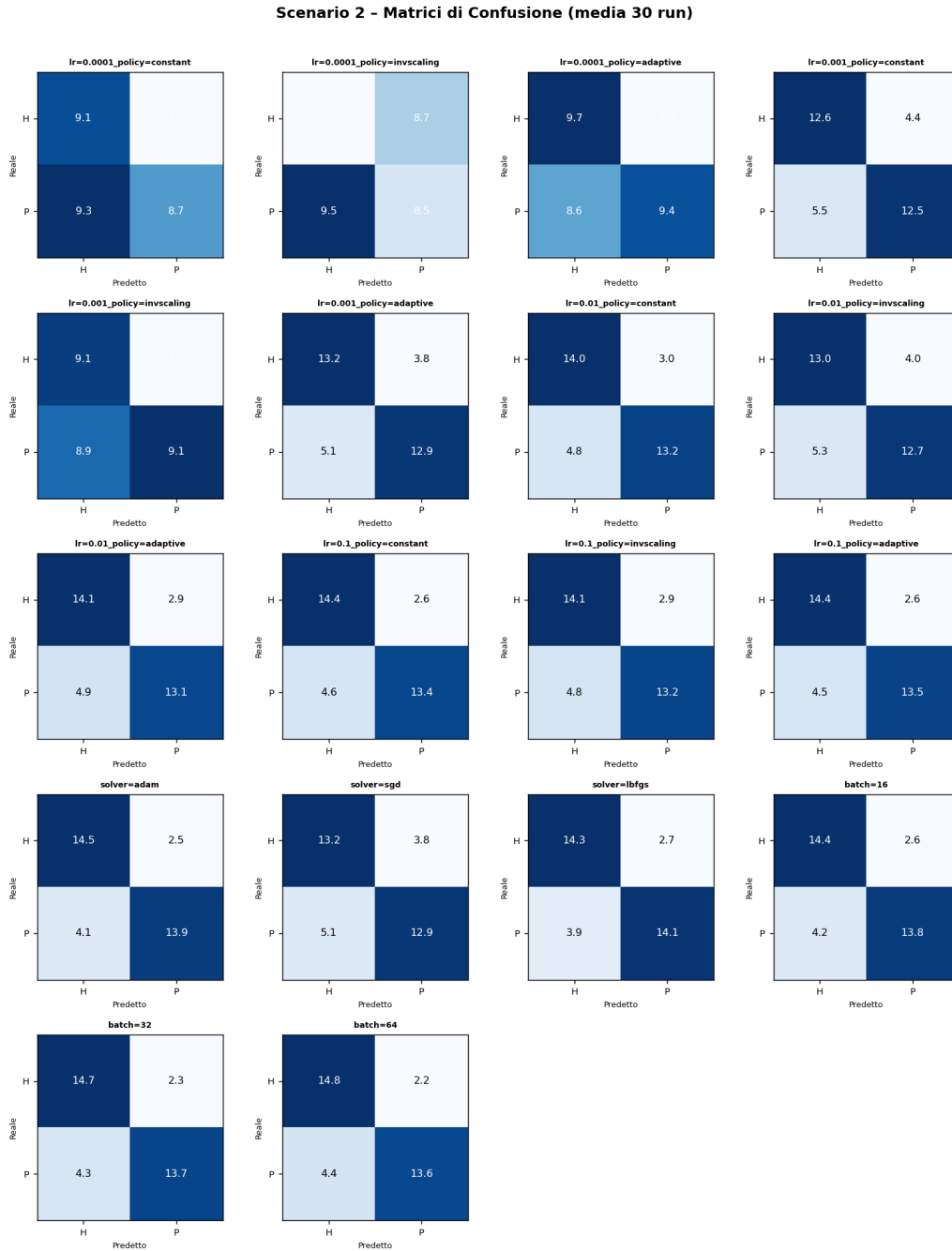


Figura 8: Scenario 2 – Matrici di confusione mediate su 30 run.

Osservazioni.

- Le configurazioni con $lr = 0.0001$ e $lr = 0.001_invscaling$ mostrano matrici quasi uniformi (ogni quadrante ≈ 9): il modello non discrimina e assegna le classi in modo sostanzialmente casuale, confermando la non-convergenza.
- A partire da $lr = 0.001_adaptive$ e soprattutto $lr = 0.01$ in poi, la diagonale torna dominante: ≈ 14 TN e ≈ 13 – 15 TP con pochi errori off-diagonale.
- Le configurazioni con **batch piccoli** (16, 32) mostrano le matrici più pulite: 14.7–14.8 TP e solo 2.2–2.3 FN, le migliori di questo scenario.
- **solver=adam** (confronto isolato) produce 14.5 TP e 4.1 FN; non migliorato dall'uso isolato rispetto alla variante combinata nei batch.

5.4 Curve di apprendimento (Loss)

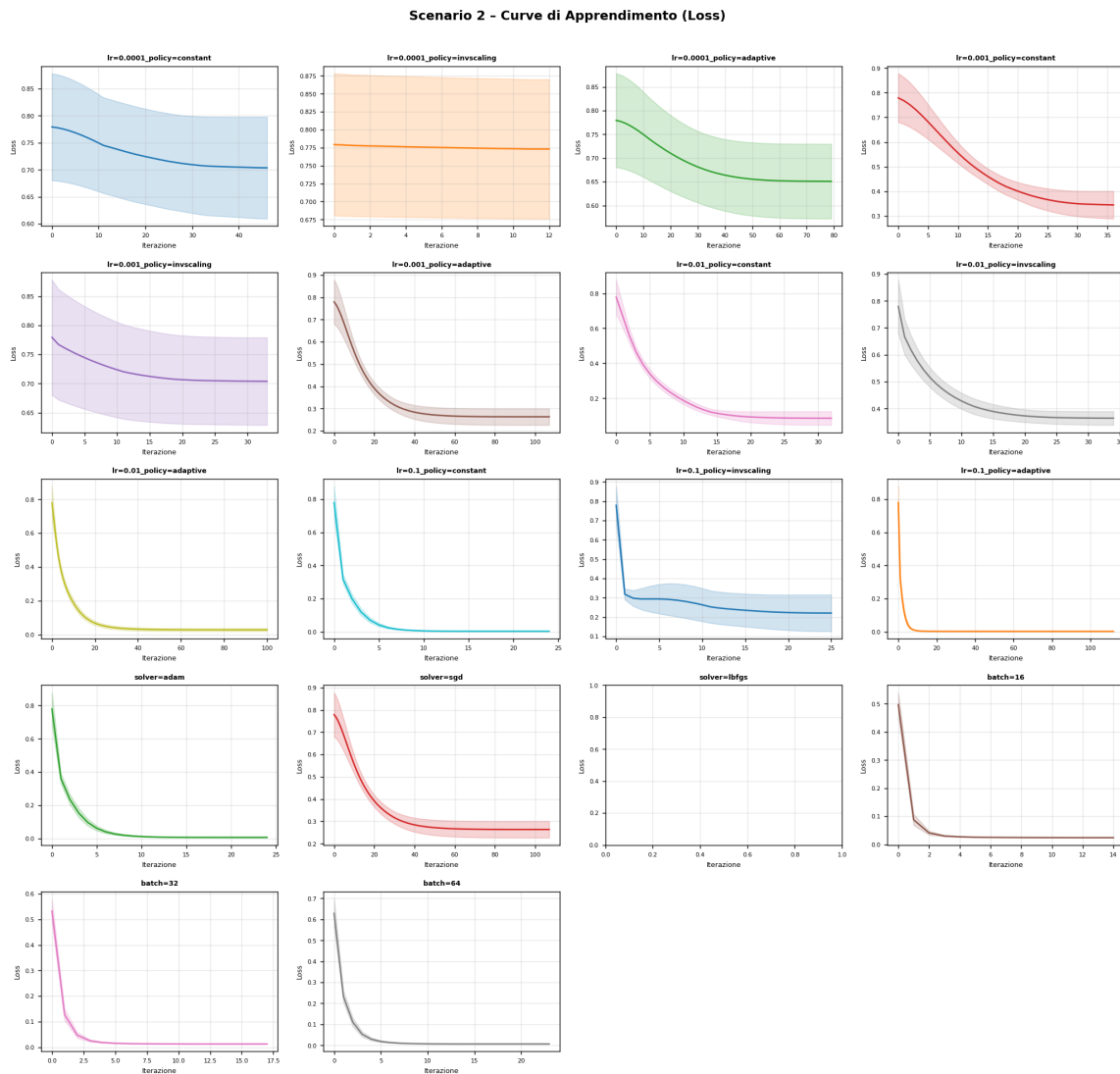


Figura 9: Scenario 2 – Curve di apprendimento (loss media $\pm$ std).

Osservazioni.

- **Non convergenza visibile:** le curve per $lr = 0.0001$ e $lr = 0.001_invscaling$ rimangono alte e piatte (≈ 0.70 – 0.85) per molte iterazioni senza scendere significativamente. È il segnale più chiaro di learning rate inadeguato.

- $lr = 0.001_adaptive$ riesce a convergere (scende fino a $\approx 0.25-0.30$), dimostrando che l'adattività del LR compensa parzialmente un valore iniziale basso.
- $lr = 0.01$ e $lr = 0.1$ convergono in modo simile e rapido (< 20 iterazioni), raggiungendo loss finali basse.
- **lbfgs** mostra curve molto lisce (quasi senza varianza) ma che si arrestano a loss relativamente alte, perché il metodo converge a un minimo locale diverso rispetto ai metodi basati su gradiente stocastico.
- **Batch size 32 e 64** convergono in < 15 iterazioni con loss finale ≈ 0 , confermando l'efficienza di Adam con batch di dimensione moderata.

5.5 Curve ROC

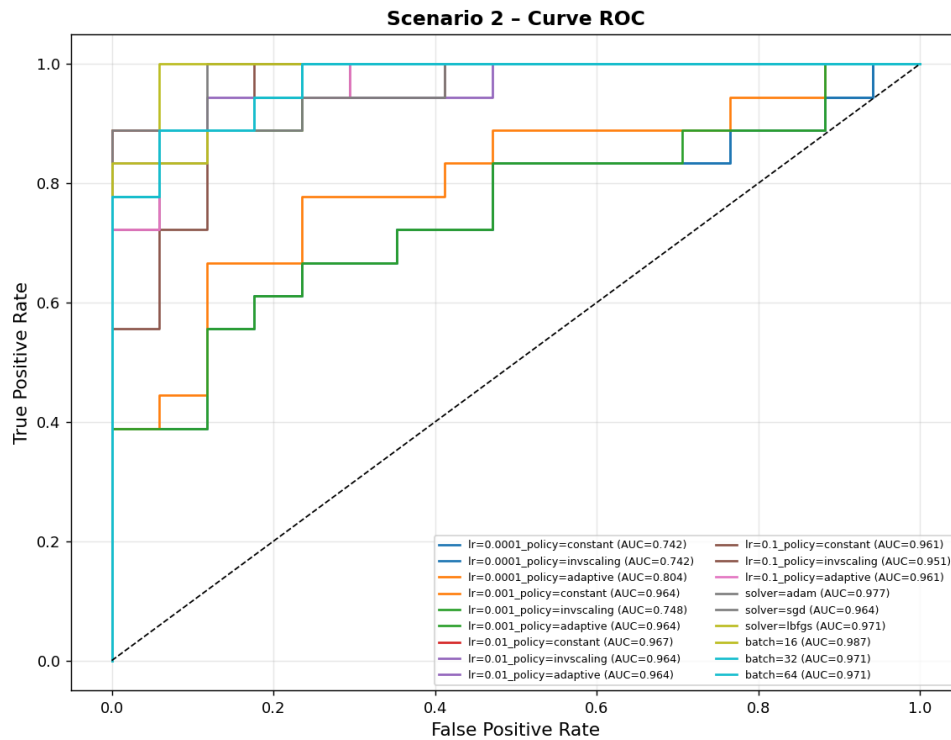


Figura 10: Scenario 2 – Curve ROC (best run per configurazione).

Osservazioni.

- Le configurazioni non convergenti ($lr = 0.0001$) mostrano $AUC \approx 0.74$, ben lontane dal potenziale del modello.
- Le configurazioni convergenti raggiungono AUC nell'intervallo 0.95–0.99, coerente con lo Scenario 1.
- **batch=16** raggiunge l'AUC più alta (0.987), probabilmente per il maggiore numero di aggiornamenti per epoca che migliora la generalizzazione.
- **solver=lbfgs** ($AUC = 0.971$) è competitivo in termini di AUC nonostante l'accuracy CV più bassa, indicando che il ranking relativo dei campioni è corretto ma la calibrazione della soglia di decisione è subottimale.

5.6 Tempi di esecuzione

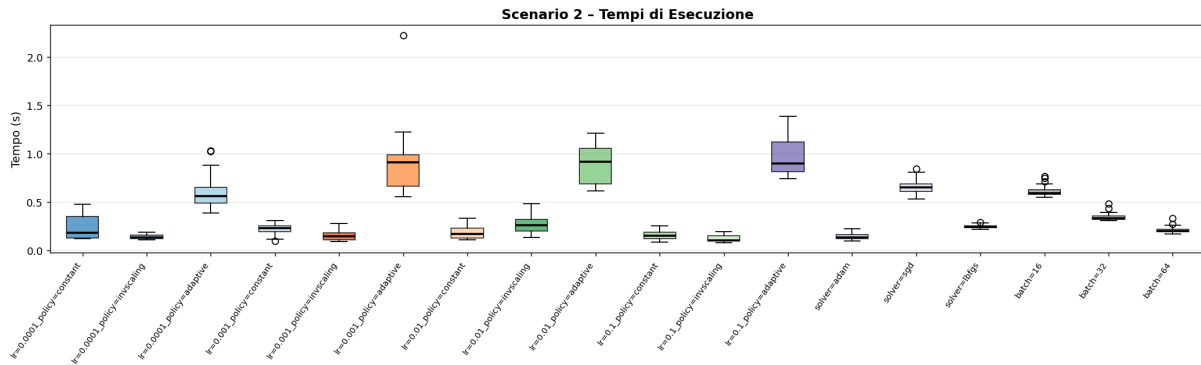


Figura 11: Scenario 2 – Tempi di esecuzione (secondi).

Osservazioni.

- Le configurazioni con $lr = 0.0001_adaptive$ e $lr = 0.001_adaptive$ e $lr = 0.01_adaptive$ sono notevolmente più lente (≈ 0.6 – 1.0 s), poiché l'adattività del learning rate permette al training di continuare per molte più iterazioni prima di attivare il criterio di stop.
- Le configurazioni con $lr = 0.0001$ (non convergenti) sono paradossalmente lente anch'esse (≈ 0.25 s): raggiungono il numero massimo di iterazioni senza convergere.
- Adam con batch piccoli** (16) è più lento di batch grandi (64) a parità di epoche, poiché esegue più aggiornamenti per epoca.
- lbfgs** mostra un tempo ridotto (≈ 0.15 s) con pochissima varianza, caratteristico dei metodi del secondo ordine che richiedono meno passi per convergere (pur raggiungendo soluzioni subottimali).

5.7 Stabilità e Overfitting

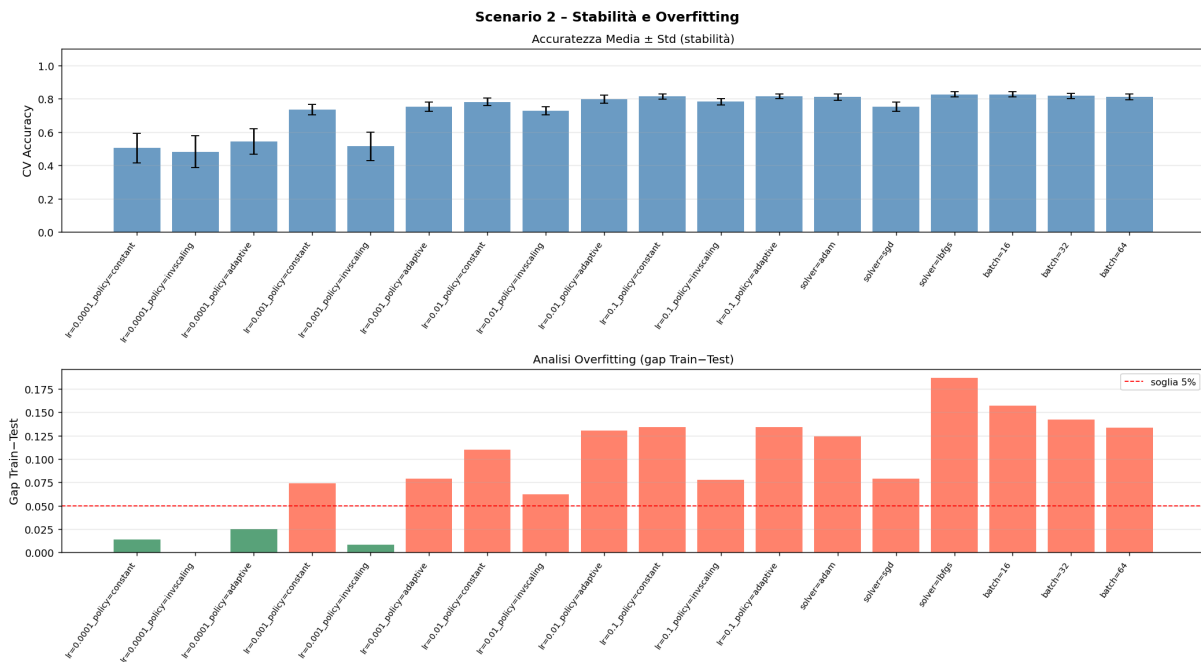


Figura 12: Scenario 2 – Stabilità (sopra) e analisi overfitting (sotto).

Osservazioni.

- **Alta instabilità per lr basso:** le prime cinque configurazioni ($lr \leq 0.001$) mostrano error bar molto grandi (fino a ± 0.10), evidenziando che l'esito dell'addestramento dipende fortemente dal seed e dall'inizializzazione quando il LR è insufficiente.
- Le configurazioni non convergenti ($lr = 0.0001$) hanno un gap Train-Test *bassissimo* (< 0.05), ma non perché generalizzino bene: semplicemente il modello non impara né in training né in test.
- A partire da $lr = 0.001_adaptive$ il gap sale sopra 0.05, indicando che la rete inizia a imparare ma produce overfitting (gap $\approx 8-18\%$ per le configurazioni convergenti).
- **solver=lbfgs** ha il gap più alto ($> 18\%$), coerente con la tendenza dei metodi quasi-Newton a memorizzare il training set su dataset di dimensione moderata.
- Tra i batch size, **batch=64** mostra il gap più contenuto ($\approx 13\%$), suggerendo che batch più grandi fungono da regolarizzatore implicito.

6 Scenario 3 – Regularizzazione ed Early Stopping

6.1 Contesto

Lo Scenario 3 fissa l'architettura (400, 200), attivazione `tanh`, solver `adam`, `lr = 0.001` e varia i parametri di regularizzazione:

1. **L2 penalty alpha:** 0.0001, 0.001, 0.01, 0.1, 0.5.
2. **Early stopping** on/off.
3. **Validation fraction:** 0.10, 0.15, 0.20.
4. **n_iter_no_change:** 5, 10, 20.

6.2 Box plot dell'accuracy (CV 5-fold)

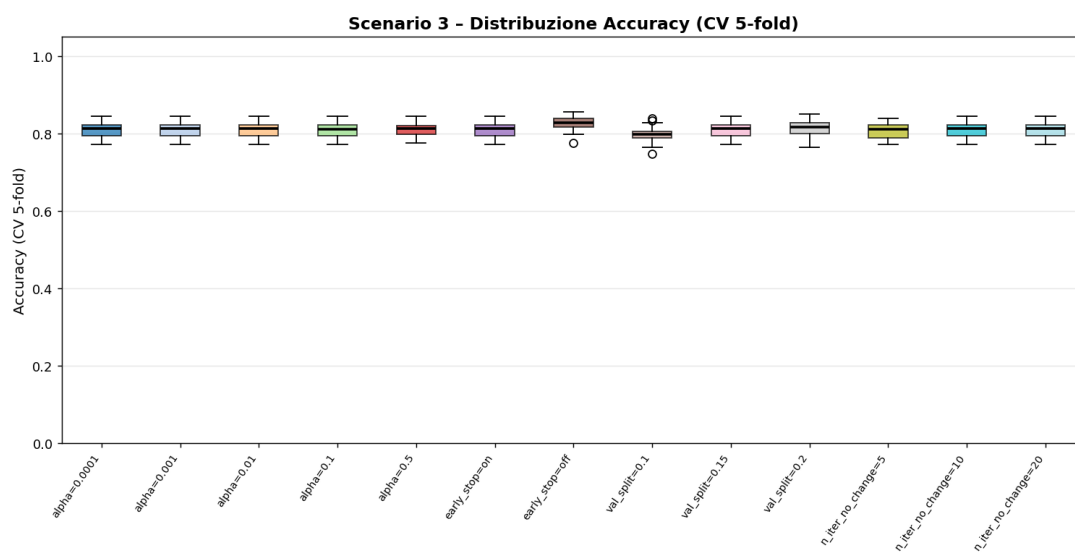


Figura 13: Scenario 3 – Distribuzione accuracy CV 5-fold.

Osservazioni.

- Questo è il grafico più “uniforme” dei tre scenari: tutte le configurazioni si attestano attorno a 0.80–0.82, con varianza molto ridotta.
- Il parametro `alpha` ha un effetto quasi nullo nell'intervallo 0.0001–0.1: le mediane sono praticamente identiche. Solo `alpha=0.5` mostra un lieve decremento (≈ 0.80), indicando che una penalizzazione così aggressiva inizia a costringere eccessivamente i pesi.
- `early_stop=off` produce la distribuzione con la mediana più alta (≈ 0.82) ma anche qualche outlier basso: senza early stopping la rete può imparare di più ma è meno stabile.
- Le variazioni di `val_split` e `n_iter_no_change` hanno impatto trascurabile sull'accuracy CV, confermando che la configurazione di base è già robusta.

Commento

L'accuracy di questo scenario è la più alta e stabile dei tre. Il dataset DARWIN è ben appreso da (400, 200) `tanh_adam` indipendentemente dai dettagli di regularizzazione, che incidono principalmente sull'overfitting (vedi Sez. 6.7).

6.3 Matrici di confusione (media 30 run)

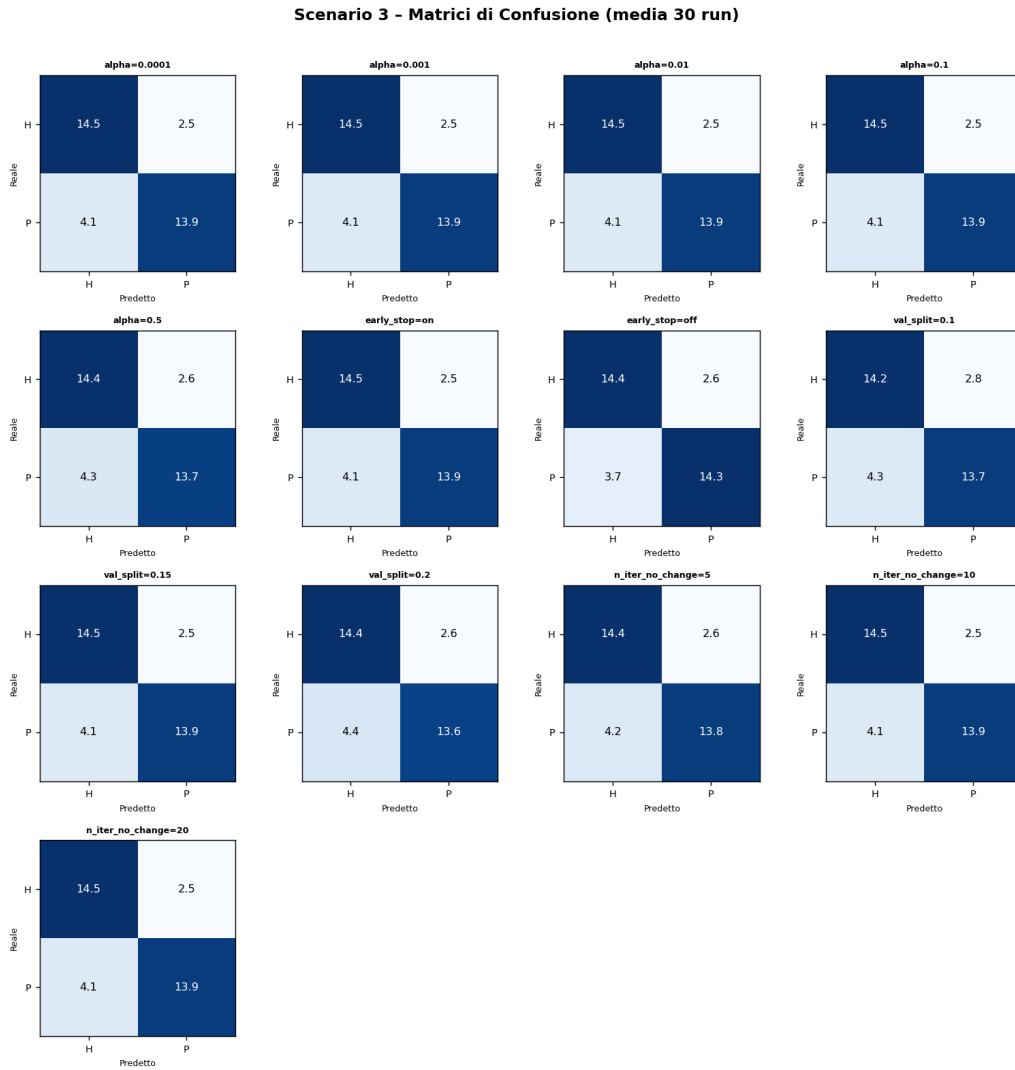


Figura 14: Scenario 3 – Matrici di confusione mediate su 30 run.

Osservazioni.

- Le matrici sono notevolmente stabili tra le configurazioni: quasi tutte mostrano 14.4–14.5 TN, 13.7–13.9 TP, 2.5–2.6 FP, 4.1–4.3 FN.
- La configurazione **early_stop=off** si distingue con soli 3.7 FN (il minimo di questo scenario), a costo però di maggiore overfitting (vedi analisi successiva).
- val_split=0.1** mostra 2.8 FP (leggermente più alto), probabilmente perché la riduzione della frazione di validazione altera il criterio di stop precoce.
- La stabilità delle matrici conferma che l'architettura e l'attivazione dominano le prestazioni, mentre la regolarizzazione fine-tuning agisce principalmente sul gap generalization.

6.4 Curve di apprendimento (Loss)

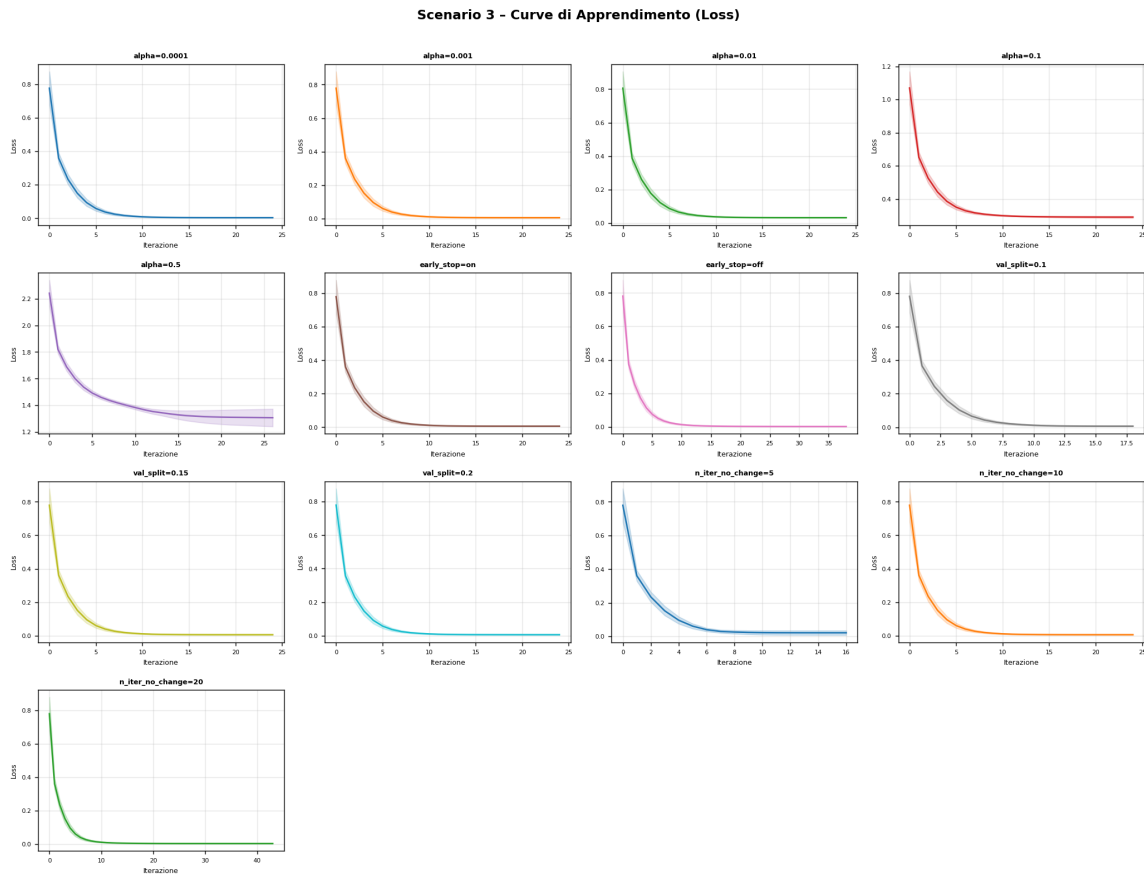


Figura 15: Scenario 3 – Curve di apprendimento (loss media $\pm$ std).

Osservazioni.

- Per **alpha** nell'intervallo 0.0001–0.1, le curve sono praticamente sovrapponibili: decrescita rapida nelle prime 5–10 iterazioni, poi convergenza a loss $\approx$ 0.03–0.05.
- **alpha=0.5** mostra una loss iniziale molto più alta ($\approx$ 2.3) e converge a un plateau più elevato ($\approx$ 1.25), confermando che la penalizzazione L2 aggressiva limita la capacità espressiva della rete.
- **early_stop=off** produce la curva più lunga (fino a 37 iterazioni) e raggiunge la loss finale più bassa, confermando che senza il criterio di arresto precoce la rete continua a ottimizzare il training set.
- **n_iter_no_change=5** arresta il training molto presto (solo 16 iterazioni in media), giustificando il tempo di esecuzione ridotto osservato nel box plot dei tempi.
- Le bande di deviazione standard sono strettissime in tutto lo Scenario 3, evidenziando la massima riproducibilità di questa configurazione base.

6.5 Curve ROC

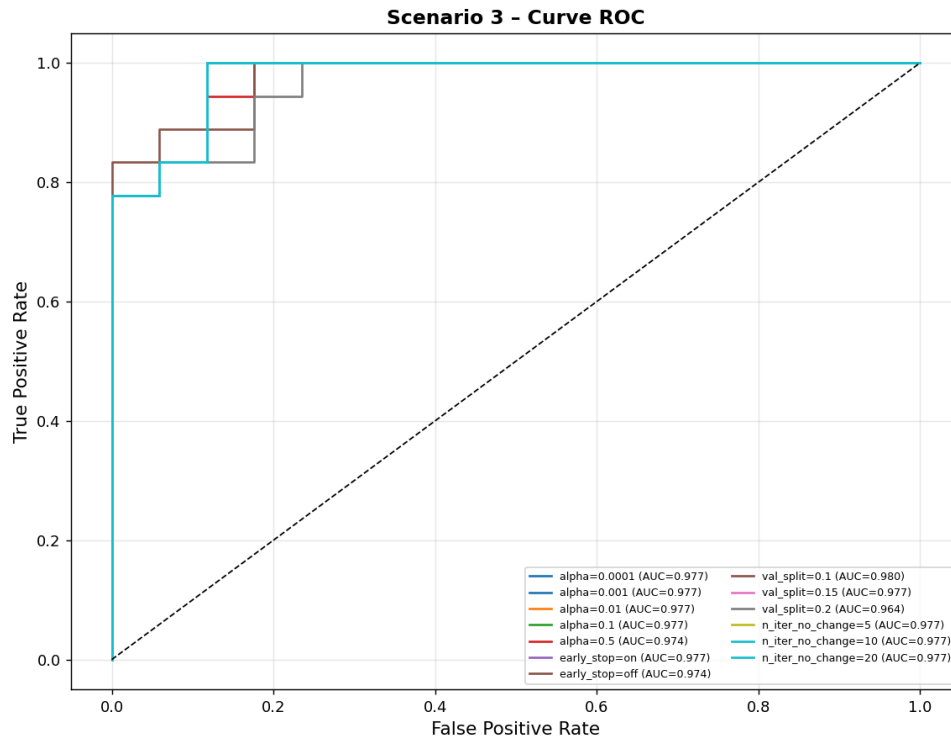


Figura 16: Scenario 3 – Curve ROC (best run per configurazione).

Osservazioni.

- Le AUC variano nel ristretto intervallo 0.964–0.980, il gruppo più compatto dei tre scenari.
- **val_split=0.1** ottiene l'AUC più alta (0.980), forse perché una validation fraction più piccola lascia più dati al training, migliorando marginalmente la qualità dei pesi finali.
- **val_split=0.2** ha l'AUC più bassa (0.964), coerentemente con il ragionamento opposto.
- Le curve ROC si sovrappongono quasi perfettamente, confermando che la capacità discriminativa del modello è insensibile alle variazioni di regolarizzazione nel range testato.

6.6 Tempi di esecuzione

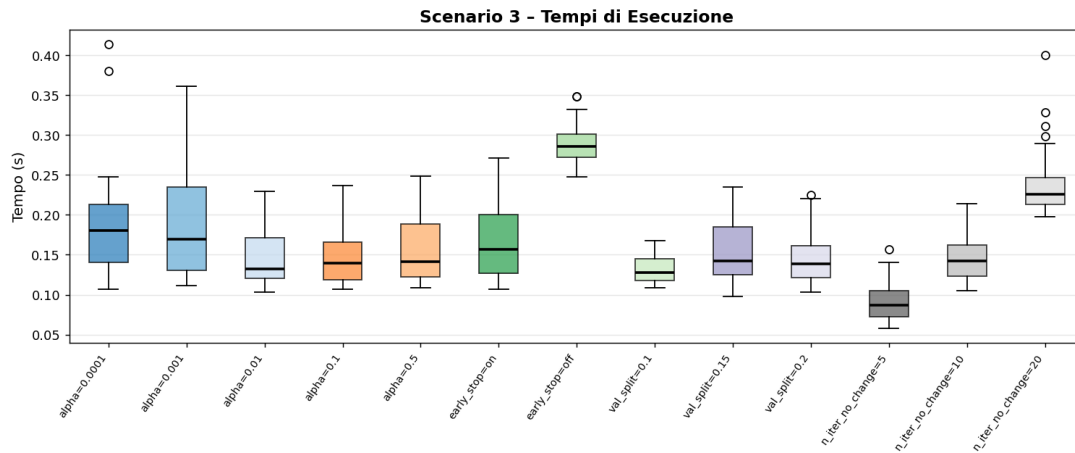


Figura 17: Scenario 3 – Tempi di esecuzione (secondi).

Osservazioni.

- I tempi sono generalmente bassi e omogenei (0.13–0.29 s), con qualche outlier per $\alpha = 0.0001$ e $\alpha = 0.001$ (fino a 0.41 s).
- **early_stop=off** è il più lento (≈ 0.29 s in mediana): senza criterio di arresto la rete percorre tutte le iterazioni disponibili.
- **n_iter_no_change=5** è il più rapido (≈ 0.09 s in mediana), poiché il training si arresta appena la loss smette di migliorare per 5 iterazioni consecutive.
- **n_iter_no_change=20** è più lento (≈ 0.23 s): il training aspetta più a lungo prima di fermarsi, permettendo affinamenti più profondi.

6.7 Stabilità e Overfitting

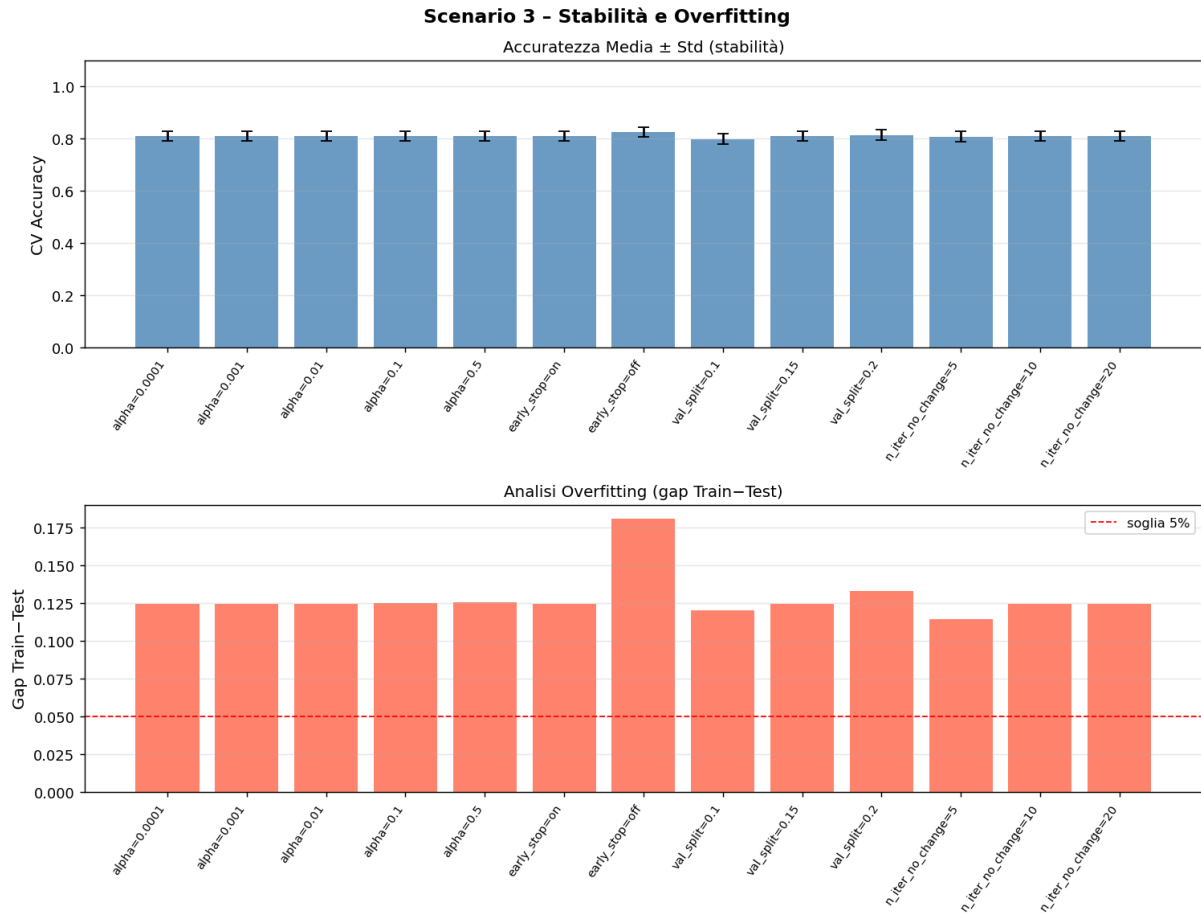


Figura 18: Scenario 3 – Stabilità (sopra) e analisi overfitting (sotto).

Osservazioni.

- Il pannello superiore conferma l'elevata stabilità: tutte le barre sfiorano 0.81 con error bar < 0.02 , indipendentemente dalla configurazione di regolarizzazione.
- Nel pannello inferiore, tutte le configurazioni superano la soglia del 5%, con gap tipici di ≈ 12 –13%.
- **early_stop=off** produce il gap più alto ($\approx 18\%$): senza early stopping la rete si adatta eccessivamente al training, confermando l'utilità del criterio di arresto precoce per il controllo dell'overfitting.
- L'aumento di **alpha** da 0.0001 a 0.5 non riduce significativamente il gap, suggerendo che per questo dataset e questa architettura la regolarizzazione L2 non è lo strumento principale di controllo dell'overfitting nell'intervallo esplorato.
- **n_iter_no_change=5** mostra il gap più basso tra le varianti di early stopping ($\approx 11\%$): terminare prima il training riduce la memorizzazione.

Commento

Nello Scenario 3 il fattore che impatta maggiormente l'overfitting è la presenza/assenza dell'early stopping, non il valore di `alpha` o di `val_split`. Abilitare l'early stopping con `n_iter_no_change ≤ 10` è la strategia di regolarizzazione più efficace per questo problema.

7 Conclusioni

7.1 Confronto Comparativo tra Scenari

La Tabella 4 riassume le migliori configurazioni per ciascun scenario secondo i criteri principali.

Tabella 4: Confronto delle migliori configurazioni per scenario.

Scenario	Migliore config.	CV Acc.	AUC	Gap OF	Tempo (s)
S1 – Architettura	(600,)_tanh	≈ 0.82	0.997	$\approx 10\%$	≈ 0.15
S2 – LR/Solver	batch=16 (adam)	≈ 0.83	0.987	$\approx 15\%$	≈ 0.60
S3 – Regulariz.	early_stop=off	≈ 0.83	0.974	$\approx 18\%$	≈ 0.29
S3 – Regol.	alpha=0.01, ES on	≈ 0.81	0.977	$\approx 12\%$	≈ 0.15

7.2 Considerazioni finali.

- L'**architettura** impatta principalmente sulla velocità di convergenza e sulla minima loss raggiungibile, non sull'accuracy CV finale che rimane in un range ristretto.
- Il **learning rate** è il parametro più critico: valori inadeguati rendono il training inutilizzabile.
- La **regolarizzazione** (L2 + early stopping) è essenziale per ridurre l'overfitting: pur non migliorando l'accuracy CV, migliora la generalizzazione.
- La configurazione **raccomandabile** complessiva è: architettura (400, 200) o (600, 300), attivazione `tanh` o `relu`, solver `adam` con `lr = 0.001`, $\alpha = 0.001$, early stopping attivo con `val_split=0.15` e `n_iter_no_change=10`. Questa configurazione bilancia accuracy (≈ 0.81), AUC (≈ 0.98), overfitting contenuto ($\approx 12\%$) e tempi ragionevoli (≈ 0.15 s).